Práctica 5
Transacciones JDBC (commit/rollback)

Venimos del CRUD con JDBC y de la reorganización por capas (Práctica 4). Ahora damos el salto a casos de uso realistas, donde una acción toca varias tablas y no podemos permitir estados intermedios.

Una transacción es "todo o nada". O se guarda todo o no se guarda nada.

Una **transacción** es decirle a la BD:
"Voy a hacer varias cosas, y **solo** quiero guardarlas si **todas** salen bien". Por ejemplo, Si realizo una transferencia bancaria: o sale dinero de A y entra en B, o no quiero que pase nada. Si hay algun error intermedio necesito **deshacer** el proceso y volver al punto justo antes de comenzar la transacción.

Para ello, en JDBC debo tener en cuenta:

- .setAutoCommit(false) → empiezas "modo transacción" desactivando el auto-commit, ya que por defecto, JDBC trabaja con auto-commit true haciendo commit automático tras cada sentencia SQL.
- **.commit()** → confirma y guarda todo lo hecho **desde que empezó la transacción** (desde que pusiste autoCommit(false))
- **.rollback()** → deshaces todo desde el inicio de la transacción.

Nota importante:

Si abres **dos conexiones**, ya no hay una única transacción. Por eso en un caso real hay que ejecutar TODO con **la misma Connection**.

Para operaciones simples (un solo SELECT, un solo INSERT) puedes seguir con repos abriendo conexión internamente como hasta ahora, pero, en cuanto quieras **transacciones** (varias sentencias coordinadas) necesitas repos que **acepten Connection**. En la práctica, lo que se hace es **tener dos versiones** de métodos:

- versión "normal" (abre su conexión) para uso simple. Ej:

```
public Optional<Libro> findById(int id) {
    String sql = "SELECT id, titulo, isbn, anio, disponible FROM libro
WHERE id = ?";
    try (Connection con = Db.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return Optional.of(mapRow(rs));
            }
            return Optional.empty();
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error buscando libro por id=" +
```

```
                id, e);
        }
    }
```

- versión “tx” para uso transaccional. El Service abre la Connection UNA vez y la pasa a todos los repos implicados como parametro. Ej:

```
public Libro findByIdOrNull(Connection con, int id) throws SQLException {

        String sql = "SELECT id, titulo, isbn, anio, disponible FROM libro
WHERE id = ?";

        try (PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);

            try (ResultSet rs = ps.executeQuery()) {

                if (!rs.next()) return null;

                return mapRow(rs);

            }

        }
```

1) Objetivos

Partiendo del proyecto de la Práctica 4 (arquitectura por capas), implementa un caso de uso que haga un cambio en BD dentro de una transacción y fuerce un error a propósito para comprobar que el rollback revierte el cambio.

- Implementar try/catch con commit/rollback usando try-with-resources.
- Comprobar con una consulta SQL que el rollback ha funcionado.
- Aprender la regla: 'si quiero transacción, NO puedo abrir conexiones nuevas dentro de Repository'.

2) Qué modificamos

- TxLibroService (Service) con un método transaccional.
- TxDemo (App) para ejecutar y comprobar por consola.
- Pequeña ampliación en JdbcLibroRepository para ejecutar SQL con Connection compartida.

3) Qué se entrega

Capturas de pantalla con las salidas por consola y la tabla en pgAdmin para el caso en el que forzamos el rollback (Punto 5) y el caso en el que comentamos el fallo para ejecutar el commit (Punto 7).

4) Pasos a seguir

1. En pgAdmin, ejecuta: SELECT id, titulo, disponible FROM libro ORDER BY id; y elige un id existente (por ejemplo 1).
2. En IntelliJ, crea/revisa com.dam.ad.db.Db con getConnection().
3. En JdbcLibroRepository, añade métodos que acepten Connection (findByIdOrNull(Connection, id), update(Connection, libro)).
4. Crea TxLibroService en com.dam.ad.service (método marcarNoDisponiblePeroFallar).
5. Crea TxDemo en com.dam.ad.app y ejecútalo.
6. Vuelve a pgAdmin y repite el SELECT: disponible debe seguir igual.

5) Código

5.1 Métodos extra en JdbcLibroRepository

Ruta: src/main/java/com/dam/ad/repository/jdbc/JdbcLibroRepository.java

Añadir (sin eliminar lo que ya funciona del CRUD):

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import com.dam.ad.model.Libro;

// Buscar por id usando Connection compartida
public Libro findByIdOrNull(Connection con, int id) throws SQLException {
    String sql = "SELECT id, titulo, isbn, anio, disponible FROM libro WHERE id =
?";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) return null;
            return mapRow(rs); // reutiliza tu mapRow(ResultSet)
        }
    }
}

// Update usando Connection compartida
public boolean update(Connection con, Libro libro) throws SQLException {
    String sql = "UPDATE libro SET titulo=?, isbn=?, anio=?, disponible=? WHERE
id=?";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, libro.getTitulo());
```

```
        ps.setString(2, libro.getIsbn());
        if (libro.getAnio() == null) ps.setObject(3, null);
        else ps.setInt(3, libro.getAnio());
        ps.setBoolean(4, libro.isDisponible());
        ps.setInt(5, libro.getId());
        return ps.executeUpdate() == 1;
    }
}
```

5.2 TxLibroService.java

Ruta: src/main/java/com/dam/ad/service/TxLibroService.java

```
package com.dam.ad.service;

import com.dam.ad.db.Db;
import com.dam.ad.model.Libro;
import com.dam.ad.repository.jdbc.JdbcLibroRepository;

import java.sql.Connection;

public class TxLibroService {

    private final JdbcLibroRepository jdbcRepo;

    public TxLibroService(JdbcLibroRepository jdbcRepo) {
        this.jdbcRepo = jdbcRepo;
    }

    public void marcarNoDisponiblePeroFallar(int idLibro) {
        try (Connection con = Db.getConnection()) {

            // 1) Empieza transacción
            con.setAutoCommit(false);



            try {
                // 2) Leer libro
                Libro l = jdbcRepo.findByIdOrNull(con, idLibro);
                if (l == null) throw new IllegalArgumentException("No existe libro
id=" + idLibro);
```

```
                // 3) Cambiar estado en el objeto
                l.setDisponible(false);

                // 4) Guardar en BD (dentro de la transacción)
                boolean ok = jdbcRepo.update(con, l);
                if (!ok) throw new IllegalStateException("No se pudo actualizar
id=" + idLibro);

                // 5) Forzar fallo
                throw new RuntimeException("Fallo forzado para demostrar
rollback");

                // 6) commit (no llegamos aquí)
                // con.commit();

            } catch (Exception e) {
                // 7) rollback si algo falla
                con.rollback();
                throw e;

            } finally {
                // 8) volver a autoCommit(true)
                con.setAutoCommit(true);
            }

        } catch (Exception e) {
            // 9) envolver en RuntimeException para no obligar a throws en App
            throw new RuntimeException("TxLibroService: se hizo rollback. Motivo:
" + e.getMessage(), e);
        }
    }
}
```

En esta parte del cocdigo forzamos un error a propósito para comprobar que el rollback funciona. Es decir: el UPDATE se llega a ejecutar, pero NO queda guardado porque no se confirma con commit.

• ¿Qué significa “forzar el fallo”? Simulamos un error que podría ocurrir en la vida real (reglas, constraints, errores de red, etc.) con una línea: throw new RuntimeException("Fallo forzado...").

• ¿Se ejecuta commit en esta demo? NO. Al lanzar la excepción con throw, el flujo salta al catch y nunca llega a con.commit().

• ¿Entonces por qué el UPDATE no se queda en la base de datos? Porque con autoCommit=false los cambios quedan “pendientes” hasta commit. En el catch hacemos con.rollback() y se deshacen.

Flujo paso a paso (lo que ocurre cuando se ejecuta marcarNoDisponiblePeroFallar):

1) Se abre UNA Connection en el service y se llama a con.setAutoCommit(false). Empieza la transacción.

2) Se lee el libro con jdbcRepo.findByIdOrNull(con, id). Ojo: se usa la MISMA Connection.

3) Se hace jdbcRepo.update(con, l). El UPDATE se ejecuta contra la BD, pero todavía no está confirmado (no hay commit).

4) Se lanza el throw new RuntimeException(...). En ese instante Java deja de ejecutar el try y salta al catch.

5) En el catch se ejecuta con.rollback(): se deshace el UPDATE y cualquier cambio hecho desde el inicio de la transacción.

6) El commit no se ejecuta porque la excepción corta el flujo antes de llegar a esa línea.

5.3 TxDemo.java

Ruta: src/main/java/com/dam/ad/app/TxDemo.java

```
package com.dam.ad.app;

import com.dam.ad.repository.jdbc.JdbcLibroRepository;
import com.dam.ad.service.TxLibroService;

public class TxDemo {
    public static void main(String[] args) {
        var repo = new JdbcLibroRepository();
        var tx = new TxLibroService(repo);

        System.out.println("=== DEMO TRANSACCION (rollback) ===");
        try {
            tx.marcarNoDisponiblePeroFallar(1);
```

```
        } catch (Exception e) {
            System.out.println("Esperado: " + e.getMessage());
        }

        System.out.println("Comprueba en pgAdmin que el libro id=1 NO se quedó
como disponible=false.");
    }
}
```

6) Problemas típicos rollback

- "No hace rollback" → casi siempre: se olvido setAutoCommit(false).
- "Rollback pero el libro sí cambió" → están usando otra Connection (repo abre una nueva).
- "current transaction is aborted…" → hubo error y siguieron lanzando SQL sin rollback.

7) Demostrar el caso "éxito" (commit):

• Comenta la línea del throw (la del fallo forzado).

• Descomenta con.commit();

• Ejecuta de nuevo TxDemo y comprueba en pgAdmin que ahora SÍ queda disponible=false (porque hubo commit).

• Vuelve a dejar el throw activado para la entrega, ya que el objetivo de la práctica es ver el rollback.